



МЕЧТАЕШЬ О КАРЬЕРЕ
УСПЕШНОГО ТРЕЙДЕРА?



MetaTrader 5

НАЧНИ С ЭТОГО ➔



MetaTrader 5 / Торговые системы

За пределами GARCH (Часть VII): Прогнозирование волатильности методом Монте-Карло в MQL5

4

Muhammad Minhas Qamar

70

Введение

В [шестой части](#) мы создали модуль моделирования, генерирующий одну ценовую траекторию MMAR. Одна траектория полезна для визуализации, но бесполезна для прогнозирования. Одна случайная реализация ничего не говорит о распределении возможных будущих состояний. Чтобы получить прогноз волатильности — точечную оценку с границами неопределенности, — нужно многократно запустить модуль моделирования и агрегировать результаты. В этом состоит метод Монте-Карло: выполнить N независимых симуляций, измерить волатильность каждой и извлечь статистические показатели из полученного распределения.

В этой статье создается класс CMonteCarlo — последний вычислительный модуль перед верхнеуровневым фасадом. Класс принимает подогнанные параметры модели, выполняет сотни или тысячи независимых симуляций MMAR на заданном горизонте прогнозирования и возвращает полный результат прогноза: среднюю волатильность, стандартное отклонение, медиану и 95-процентный доверительный интервал. Результат отвечает на важный для торговли вопрос: "Какова ожидаемая волатильность на следующих N барах и насколько мы в этом уверены?"

Мы рассмотрим следующие темы:

1. [Архитектура CMonteCarlo](#)
2. [Адаптивная глубина каскада](#)
3. [Цикл моделирования](#)
4. [Статистические характеристики прогноза](#)
5. [Запуск полного пайплайна](#)
6. [Заключение](#)

Архитектура CMonteCarlo

Модуль Монте-Карло расположен на верхнем уровне генеративного стека. Модуль получает результаты четвертой части (показатель Херста, выборочная волатильность) и пятой части (тип распределения и параметры), запускает несколько экземпляров модуля моделирования из шестой части и возвращает единую структуру MMARForecast. Модуль подключает файлы MMARConstants.mqh (общие структуры данных) и SimulationEngine.mqh. Файл SimulationEngine.mqh, в свою очередь, подключает заголовочные файлы Стандартной библиотеки для FFT, разложения Холецкого и генерации случайных чисел, рассмотренные в шестой части. Сам класс намеренно сделан компактным: это скорее оркестратор, чем вычислитель.

```
//+-----+  
//| CMonteCarlo class |
```



```
//+-----+
class CMonteCarlo
{
private:
    //--- Configuration
    int          m_n_simulations;
    int          m_forecast_horizon;
    int          m_cascade_b;

    //--- Model parameters (from fitting stage)
    double       m_H;
    ENUM_MMAR_DISTRIBUTION m_dist_type;
    double       m_dist_params[4];
    double       m_sample_volatility;

    //--- Results
    double       m_vol_forecasts[];
    MMARForecast m_forecast;
    bool         m_ran;

    //--- Internal helpers
    int          CalculateCascadeDepth(int forecast_length);
    double       ComputeVolatility(const double &returns[], int n);

public:
    CMonteCarlo(void);
    ~CMonteCarlo(void);

    //--- Initialization
    bool         Init(double H, ENUM_MMAR_DISTRIBUTION dist_type,
                     const double &dist_params[], double sample_volatility,
                     int n_simulations, int forecast_horizon, int cascade_b = 2);

    //--- Execution
    bool         Run(void);

    //--- Results access
    MMARForecast GetForecast(void) { return m_forecast; }
    void         GetAllForecasts(double &forecasts[]);
    bool         HasRun(void) { return m_ran; }
};
```

Интерфейс прост: Init() принимает параметры модели и конфигурацию моделирования, Run() выполняет все симуляции, затем GetForecast() возвращает агрегированный результат. Метод GetAllForecasts() предоставляет доступ к необработанным значениям волатильности каждой отдельной симуляции. Это полезно для построения пользовательских распределений и диагностики, выходящей за пределы стандартной структуры прогноза.

Метод Init() проверяет все входные данные и заранее выделяет память для массива результатов:

```
//+-----+
//| Init - Configure simulation parameters |
//+-----+
bool CMonteCarlo::Init(double H, ENUM_MMAR_DISTRIBUTION dist_type,
                      const double &dist_params[], double sample_volatility,
                      int n_simulations, int forecast_horizon, int cascade_b)
{
```



```

if(H <= 0.0 || H >= 1.0)
{
    PrintFormat("MonteCarlo::Init - H=%.4f out of range (0,1)", H);
    return false;
}
if(n_simulations < 1)
{
    Print("MonteCarlo::Init - need at least 1 simulation");
    return false;
}
if(forecast_horizon < 1)
{
    Print("MonteCarlo::Init - forecast_horizon must be >= 1");
    return false;
}
if(sample_volatility <= 0.0)
{
    Print("MonteCarlo::Init - sample_volatility must be > 0");
    return false;
}

m_H = H;
m_dist_type = dist_type;
m_sample_volatility = sample_volatility;
m_n_simulations = n_simulations;
m_forecast_horizon = forecast_horizon;
m_cascade_b = MathMax(2, cascade_b);

for(int i = 0; i < 4; i++)
    m_dist_params[i] = (i < ArraySize(dist_params)) ? dist_params[i] : 0.0;

ArrayResize(m_vol_forecasts, n_simulations);
ArrayInitialize(m_vol_forecasts, 0.0);

m_ran = false;
return true;
}

```

Параметр `forecast_horizon` заслуживает особого внимания. Он задает, на сколько баров вперед нужно прогнозировать волатильность. Для данных M10 7 200 баров соответствуют 7 200 × 10 минутам, то есть примерно 50 дням. Модуль моделирования должен генерировать траектории не короче этой длины. Это влияет на расчет глубины каскада — тему следующего раздела.

Адаптивная глубина каскада

Модуль моделирования из шестой части генерирует траектории длиной b^k (основание каскада в степени глубины каскада). Поскольку горизонт прогнозирования может содержать произвольное число баров, нужно вычислить минимальную глубину каскада k , обеспечивающую не менее `forecast_horizon + 1` точек:

```

//+-----+
//| CalculateCascadeDepth - Find k such that b^k >= forecast_length |
//+-----+
int CMonteCarlo::CalculateCascadeDepth(int forecast_length)
{

```



```
return (int)MathCeil(MathLog((double)(forecast_length + 1)) / MathLog((double)m_casc
}
```

Для нашего горизонта прогнозирования в 7 200 баров при $b=2$ это дает $\text{ceil}(\log_2(7201)) = \text{ceil}(12.81) = 13$. Следовательно, каждая симуляция сгенерирует $2^{13} = 8192$ точки. Для расчета волатильности затем используются только первые 7 200 доходностей каждой траектории. Избыточные точки ($8192 - 7200 = 992$) просто отбрасываются. Это эффективнее, чем точно подгонять каскад под горизонт: бинарные каскады дают только степени числа 2, а накладные расходы на генерацию нескольких дополнительных точек незначительны по сравнению с затратами на построение каскада и FBM.

Выбор k напрямую влияет на производительность. При $k=13$ каждая симуляция генерирует 8 192 точки и использует для FBM метод Дэвиса-Харта, поскольку $n > 1,024$. При $k=10$ использовался бы метод Холецкого: 1 024 точки подходят для коротких горизонтов. Модуль моделирования автоматически выбирает метод без дополнительных действий со стороны пользователя.

На рисунке 1 показан этот процесс выбора. Начиная с $k=10$, каждая следующая степень двойки остается меньше горизонта в 7 200 баров, пока при $k=13$ не получится 8 192 точки — достаточно, чтобы превысить целевой горизонт на 992 точки. После расчета волатильности избыточные точки просто отбрасываются; связанные с этим накладные расходы незначительны по сравнению с затратами на построение каскада и FBM.

Made By MMQ

Рис. 1. Адаптивная глубина каскада: степени двойки сравниваются с горизонтом прогнозирования, пока b^k не станет достаточно большим; избыточные точки отбрасываются.

Цикл моделирования

Метод `Run()` — ядро модуля Монте-Карло. Для каждого прогона метод создает новый модуль моделирования, генерирует траекторию, извлекает доходности на горизонте прогнозирования и рассчитывает волатильность:

```
//+-----+
//| Run - Execute all simulations and compute forecast statistics |
//+-----+
bool CMonteCarlo::Run(void)
```



```

{
    if(m_n_simulations == 0)
    {
        Print("MonteCarlo::Run - not initialized");
        return false;
    }

    int cascade_k = CalculateCascadeDepth(m_forecast_horizon);
    int n_points = (int)MathPow(m_cascade_b, cascade_k);

    PrintFormat("MonteCarlo: %d simulations, horizon=%d bars, k=%d, N=%d",
        m_n_simulations, m_forecast_horizon, cascade_k, n_points);

    int n_success = 0;
    int n_fail = 0;

    uint t0 = GetTickCount();

    //--- Run each simulation
    for(int sim = 0; sim < m_n_simulations; sim++)
    {
        CSimulationEngine engine;
        if(!engine.Init(m_H, m_dist_type, m_dist_params, m_sample_volatility,
            m_cascade_b, cascade_k))
        {
            n_fail++;
            continue;
        }

        if(!engine.Generate())
        {
            n_fail++;
            continue;
        }

        double sim_returns[];
        int n_sim_returns;
        engine.GetReturns(sim_returns, n_sim_returns);

        int n_use = MathMin(m_forecast_horizon, n_sim_returns);
        double vol = ComputeVolatility(sim_returns, n_use);
        m_vol_forecasts[n_success] = vol;
        n_success++;
    }

    uint elapsed = GetTickCount() - t0;

    if(n_success == 0)
    {
        Print("MonteCarlo::Run - all simulations failed");
        return false;
    }

    if(n_fail > 0)
        PrintFormat("MonteCarlo: %d/%d succeeded (%d failed)",
            n_success, m_n_simulations, n_fail);
}

```

```

    ArrayResize(m_vol_forecasts, n_success);

    // ... statistics computation follows ...
}

```

При каждой итерации создается новый экземпляр CSimulationEngine. Это сделано намеренно: модуль хранит сгенерированные данные в массивах-членах класса, поэтому повторное использование экземпляра потребовало бы явной очистки между прогонами. Создавать новый объект при каждом прогоне — это более чистое и безопасное решение; затраты на выделение памяти незначительны по сравнению с генерацией каскада и FBM внутри него.

Счетчик сбоев `n_fail` учитывает редкие граничные случаи, когда модуль моделирования завершается с ошибкой. Например, метод Холецкого может завершиться с ошибкой, а резервный метод Дэвиса-Харта — также не сработает при вырожденных входных данных. Вместо прерывания всего моделирования Монте-Карло неудачные симуляции просто пропускаются, а число успешных сообщается отдельно. Итоговый прогноз рассчитывается только по успешным прогонам, а размер массива результатов корректируется соответственно.

Вспомогательная функция `ComputeVolatility` вычисляет стандартное отклонение по первым `n_use` доходностям каждой смоделированной траектории:

```

//+-----+
//| ComputeVolatility - Standard deviation of a return series      |
//+-----+
double CMonteCarlo::ComputeVolatility(const double &returns[], int n)
{
    if(n <= 0)
        return 0.0;

    double sum = 0.0;
    double sum_sq = 0.0;
    for(int i = 0; i < n; i++)
    {
        sum += returns[i];
        sum_sq += returns[i] * returns[i];
    }
    double mean = sum / n;
    double variance = sum_sq / n - mean * mean;
    return MathSqrt(MathMax(variance, 0.0));
}

```

Это стандартное отклонение генеральной совокупности: деление выполняется на n , а не на $n-1$. При большом n (в данном случае — 7 200) разница между стандартным отклонением генеральной совокупности и выборочным стандартным отклонением пренебрежимо мала. Проверка `MathMax` предотвращает извлечение квадратного корня из отрицательного числа, возможное из-за потери значимости при вычислениях с плавающей точкой, когда дисперсия крайне мала.

На рисунке 2 показан процесс моделирования Монте-Карло. Сначала моделируется одна траектория MMAR, затем из той же начальной точки генерируется множество независимых траекторий. На правой панели значения волатильности отдельных траекторий представлены в виде гистограммы, показывающую распределение прогноза. Среднее и медиана почти совпадают, что указывает на симметричность распределения. 95-процентный доверительный интервал охватывает основную массу результатов, а траектории за его пределами на левой панели показаны приглушенно.

Рис. 2. Метод Монте-Карло в действии: независимые траектории MMAR расходятся из одной начальной точки, для каждой измеряется волатильность, а результаты агрегируются в распределение прогноза с границами неопределенности.

Статистика прогноза

После завершения всех симуляций получаем массив значений волатильности — по одному для каждой успешной симуляции. Во второй части метода Run() этот массив сортируется, после чего из него извлекается статистика прогноза:

```
//--- Sort for percentile statistics
double sorted[];
ArrayResize(sorted, n_success);
ArrayCopy(sorted, m_vol_forecasts);
ArraySort(sorted);

//--- Mean and standard deviation
double sum = 0.0;
double sum_sq = 0.0;
for(int i = 0; i < n_success; i++)
{
    sum += sorted[i];
    sum_sq += sorted[i] * sorted[i];
}
double mean = sum / n_success;
double variance = sum_sq / n_success - mean * mean;
double std_dev = MathSqrt(MathMax(variance, 0.0));

//--- Median
double median;
if(n_success % 2 == 0)
    median = (sorted[n_success / 2 - 1] + sorted[n_success / 2]) / 2.0;
else
    median = sorted[n_success / 2];
```

```
//--- 95% confidence interval (percentile method)
int ci_lo_idx = (int)MathFloor(0.025 * n_success);
int ci_hi_idx = (int)MathFloor(0.975 * n_success);
ci_lo_idx = MathMax(0, MathMin(ci_lo_idx, n_success - 1));
ci_hi_idx = MathMax(0, MathMin(ci_hi_idx, n_success - 1));

m_forecast.mean_volatility = mean;
m_forecast.std_volatility = std_dev;
m_forecast.median_volatility = median;
m_forecast.ci_lower = sorted[ci_lo_idx];
m_forecast.ci_upper = sorted[ci_hi_idx];
m_forecast.n_simulations = n_success;
m_forecast.forecast_horizon = m_forecast_horizon;

PrintFormat("MonteCarlo complete: %d sims in %u ms", n_success, elapsed);
PrintFormat("  Mean vol: %.6f | Std: %.6f | Median: %.6f",
    mean, std_dev, median);
PrintFormat("  95%% CI: [%.6f, %.6f]",
    m_forecast.ci_lower, m_forecast.ci_upper);

m_ran = true;
return true;
}
```

Для 95%-ного доверительного интервала используется метод процентилей, а не параметрический подход (среднее $\pm 1,96 \cdot$ стандартное отклонение). Этот подход более устойчив, поскольку не предполагает определенной формы распределения волатильности. При 100 симуляциях 2,5-й процентиль соответствует второму наименьшему значению, а 97,5-й — 97-му. При 1 000 симуляциях разрешение повышается до 25-го и 975-го значений.

Различие между средним и медианой важно для интерпретации. Если распределение волатильности, полученное в результате симуляций, симметрично, эти значения почти совпадают. Если каскад изредка генерирует аномальные траектории с крайне высокой волатильностью, что возможно при распределениях множителей с тяжелыми хвостами, например Gamma, среднее смещается вверх, тогда как медиана остается устойчивой. Для принятия торговых решений медиана часто служит более надежной оценкой.

Запуск полного пайплайна

Теперь можно написать тестовый скрипт, объединяющий все три аналитических этапа — анализ разбиений, подгонку спектра и прогнозирование методом Монте-Карло — в единый сквозной пайплайн. Это полный рабочий процесс MMAR — от исходных ценовых данных до прогноза волатильности:

```
//+-----+
//|                                     MMAR MonteCarlo Test |
//|                                     Copyright 2026, MMQ - Muhammad Minhas Qamar |
//|                                     https://www.mql5.com/ru/articles/22537 |
//+-----+
#property copyright "Copyright 2026, MMQ - Muhammad Minhas Qamar"
#property link      "https://www.mql5.com/ru/articles/22537"
#property version   "1.00"
#property strict
#property script_show_inputs

#include <MMAR\PartitionAnalysis.mqh>
```



```

#include <MMAR\SpectrumFitter.mqh>
#include <MMAR\MonteCarlo.mqh>

input int      InpBars          = 50000;
input int      InpForecastBars = 7200;    // ~50 days of M10
input int      InpSimulations   = 100;    // Start small, increase to 1000
input int      InpCascadeB      = 2;
input int      InpSeed          = 42;

input double   InpQMin          = 0.01;
input double   InpQMax          = 30.0;
input double   InpQStep         = 0.5;
input int      InpDtMin         = 1;
input int      InpDtMax         = 150;
input double   InpDtSpacing     = 1.1;

//+-----+
//| Script program start function |
//+-----+
void OnStart()
{
    MathSrand(InpSeed);

    PrintFormat("=== MMAR Monte Carlo Test ===");
    PrintFormat("Symbol: %s | Period: %s", _Symbol, EnumToString(_Period));
    PrintFormat("Forecast: %d bars | Simulations: %d",
                InpForecastBars, InpSimulations);

    //-- Load close prices and compute log returns
    double close[];
    int copied = CopyClose(_Symbol, _Period, 0, InpBars, close);
    if(copied < 1000)
        { PrintFormat("Not enough bars: %d", copied); return; }
    PrintFormat("Loaded %d bars", copied);

    double returns[];
    int n_returns = copied - 1;
    ArrayResize(returns, n_returns);
    for(int i = 0; i < n_returns; i++)
        returns[i] = MathLog(close[i + 1] / close[i]);

    //-- Stage 1: Partition Analysis
    uint t0 = GetTickCount();

    CPartitionAnalysis partition;
    partition.SetPartitionConfig(InpDtMin, InpDtMax, InpDtSpacing);
    partition.SetMomentConfig(InpQMin, InpQMax, InpQStep);

    if(!partition.Init(returns, n_returns))
        { Print("Partition Init failed"); return; }
    if(!partition.Analyze())
        { Print("Partition Analyze failed"); return; }

    double H = partition.GetH();
    double sample_vol = partition.GetSampleVolatility();
    uint t_partition = GetTickCount() - t0;

```

```

PrintFormat("\n--- Partition (%u ms) ---", t_partition);
PrintFormat("H = %.4f | Vol = %.6f | Score = %d/9",
            H, sample_vol, partition.GetMultifractalScore());

//--- Stage 2: Spectrum Fitting
t0 = GetTickCount();

double tau[], q_vals[];
partition.GetTauQ(tau, q_vals);
int n_q = partition.GetNumQ();

CSpectrumFitter fitter;
if(!fitter.Init(q_vals, tau, n_q, H))
    { Print("SpectrumFitter Init failed"); return; }
if(!fitter.ComputeSpectrum())
    { Print("ComputeSpectrum failed"); return; }
if(!fitter.FitAllDistributions())
    { Print("FitAllDistributions failed"); return; }

uint t_spectrum = GetTickCount() - t0;

ENUM_MMAR_DISTRIBUTION best_dist = fitter.GetBestDistribution();
double dist_params[];
fitter.GetBestParams(dist_params);

string dist_names[] = {"Normal", "Binomial", "Poisson", "Gamma"};
PrintFormat("\n--- Spectrum (%u ms) ---", t_spectrum);
PrintFormat("Best: %s | SSE = %.6f", dist_names[(int)best_dist], fitter.GetBestSSE());

//--- Stage 3: Monte Carlo
t0 = GetTickCount();

CMonteCarlo mc;
if(!mc.Init(H, best_dist, dist_params, sample_vol,
            InpSimulations, InpForecastBars, InpCascadeB))
    { Print("MonteCarlo Init failed"); return; }

if(!mc.Run())
    { Print("MonteCarlo Run failed"); return; }

uint t_mc = GetTickCount() - t0;

//--- Print forecast
MMARForecast forecast = mc.GetForecast();

PrintFormat("\n--- Forecast Results ---");
PrintFormat("Horizon:          %d bars", forecast.forecast_horizon);
PrintFormat("Simulations:          %d", forecast.n_simulations);
PrintFormat("Mean volatility: %.6f", forecast.mean_volatility);
PrintFormat("Std volatility: %.6f", forecast.std_volatility);
PrintFormat("Median:            %.6f", forecast.median_volatility);
PrintFormat("95%% CI:           [%.6f, %.6f]", forecast.ci_lower, forecast.ci_upper);
PrintFormat("Sample vol:        %.6f (historical)", sample_vol);
PrintFormat("Ratio:             %.2f", forecast.mean_volatility / sample_vol);

//--- Timing summary
PrintFormat("\n--- Timing ---");

```

```

PrintFormat("Partition:    %u ms", t_partition);
PrintFormat("Spectrum:    %u ms", t_spectrum);
PrintFormat("Monte Carlo: %u ms (%d sims)", t_mc, InpSimulations);
PrintFormat("Total:      %u ms", t_partition + t_spectrum + t_mc);

Print("\n=== Monte Carlo Test Complete ===");
}

```

Для EURUSD на таймфрейме M10 (50 000 баров), при горизонте в 7 200 баров (около 50 дней) и 100 симуляциях скрипт выдает следующие результаты. Обратите внимание: модуль подгонки спектра и модуль Монте-Карло выводят внутреннюю диагностику — значения SSE распределений и выбор глубины каскада. Поэтому будет доступно больше деталей, чем явно выводит тестовый скрипт:

```

=== MMAR Monte Carlo Test ===
Symbol: EURUSD | Period: PERIOD_M10
Forecast: 7200 bars | Simulations: 100 | Seed: 42
Loaded 50000 bars
Multifractality check: concavity=1 linearity=1 variation=1 total=3/9

--- Partition (422 ms) ---
H = 0.4817 | Vol = 0.000419 | Score = 3/9
SpectrumFitter: 58 spectrum points, alpha=[0.1351, 0.5149], peak at alpha_0=0.5149 f=0.
Fitting spectrum to 4 distributions...
  Normal:    SSE=69.032584  alpha_0=0.514942
  Binomial:  SSE=0.123991  alpha_min=0.188628  alpha_max=0.772413
  Poisson:   SSE=10.212068  alpha_0=0.514942
  Gamma:     SSE=0.178587  alpha_0=0.514942  gamma=1.244125
BEST FIT: Binomial (SSE=0.123991, RMSE=0.046236)

--- Spectrum (16 ms) ---
Best: Binomial | SSE = 0.123991
MonteCarlo: 100 simulations, horizon=7200 bars, k=13, N=8192
MonteCarlo complete: 100 sims in 875 ms
  Mean vol: 0.000338 | Std: 0.000009 | Median: 0.000339
  95% CI: [0.000322, 0.000356]

--- Forecast Results ---
Horizon:      7200 bars
Simulations:  100
Mean volatility: 0.000338
Std volatility: 0.000009
Median:       0.000339
95% CI:       [0.000322, 0.000356]
Sample vol:   0.000419 (historical)
Ratio:        0.81

--- Timing ---
Partition:    422 ms
Spectrum:     16 ms
Monte Carlo:  875 ms (100 sims)
Total:        1313 ms

=== Monte Carlo Test Complete ===

```

Результаты дают убедительную картину. MMAR прогнозирует среднюю волатильность 0,000338 на следующие 7 200 баров, что на 19% ниже исторической выборочной волатильности 0,000419 (отношение

= 0,81). Это не ошибка: модель прогнозирует, что будущая волатильность будет ниже, чем в недавнем прошлом. 95%-ный доверительный интервал [0,000322; 0,000356] чрезвычайно узок: стандартное отклонение между симуляциями составляет всего 0,000009. Такая плотная группировка согласуется со структурой MMAR: параметры каскада определяются долгосрочными свойствами масштабирования, а не краткосрочными всплесками волатильности.

Среднее и медиана почти совпадают (0,000338 против 0,000339), что подтверждает симметричность распределения прогноза: аномальные траектории с большими значениями не искажают среднее. Если бы распределение каскадного множителя имело тяжелые хвосты, например Gamma с малым параметром формы, среднее могло бы смещаться вверх под влиянием редких экстремальных траекторий.

С точки зрения производительности весь пайплайн выполняется за 1,3 секунды: 422 мс требуется на анализ разбиений, 16 мс — на подгонку спектра и 875 мс — на 100 симуляций Монте-Карло. При большем числе симуляций этап Монте-Карло занимает основную часть времени выполнения. При 100 симуляциях построение каждой траектории занимает около 8,75 мс (метод Дэвиса-Харта при $k=13$ и $N=8192$). Выполнение 1 000 симуляций заняло бы примерно 8,75 секунды — этого достаточно для периодической перекалибровки советника в реальной торговле, но не для запуска на каждом тике.

Заключение

В этой статье мы создали класс CMonteCarlo, завершив разработку генеративной части библиотеки MMAR. Модуль получает подогнанные параметры аналитических этапов (части 4 и 5) и модуль моделирования из шестой части, выполняет множество независимых симуляций траекторий MMAR и формирует распределение прогнозов волатильности с количественной оценкой неопределенности.

- Адаптивная глубина каскада. Автоматически выбирает минимальное k , при котором b^k покрывает горизонт прогнозирования, обеспечивая баланс между разрешением и вычислительными затратами.
- Надежная обработка сбоев. Позволяет продолжать моделирование Монте-Карло при сбоях отдельных симуляций; в итоговой статистике учитываются только успешные прогоны.
- Доверительные интервалы на основе процентилях. Не требуют предположений о распределении прогнозируемой волатильности и дают реалистичные границы неопределенности.
- Производительность полного пайплайна. Выполнение 100 симуляций за 1,3 секунды делает MMAR практичным инструментом для периодической перекалибровки в работающей торговой системе.

Теперь готовы все четыре вычислительных модуля: анализ разбиений, подгонка спектра, модуль моделирования и прогнозирование методом Монте-Карло. В восьмой части мы объединим их с помощью верхнеуровневого класса-фасада CMMAR и используем готовую библиотеку в рабочем советнике, который подгоняет модель по историческим данным и формирует прогнозы волатильности для торговых решений.

Программы, представленные в этой статье, предназначены только для образовательных целей. Они не предназначены для реальной торговли и не оптимизированы для эффективной работы в реальных рыночных условиях. Этот код помогает понять прогнозирование методом Монте-Карло с мультифрактальными моделями; прежде чем адаптировать его для реальной торговли, всегда тщательно проверяйте его на демо-счетах.

Получение исходного кода через MQL5 Algo Forge

Все исходные файлы прикреплены ниже к этой статье, но полный репозиторий также доступен на [MQL5 Algo Forge](#) — платформе сообщества на базе Git для публикации торговых проектов и совместной работы над ними.

Имя файла	Описание
MQL5\Include\MMAR\MMARConstants.mqh	Общие перечисления, структуры и определения типов библиотеки MMAR (часть 4)
MQL5\Include\MMAR\OLS.mqh	Регрессия методом наименьших квадратов через псевдообратную матрицу на основе SVD с полным набором диагностических показателей (часть 4)
MQL5\Include\MMAR\HurstGHE.mqh	Оценщик обобщенного показателя Херста, резервный метод (часть 4)
MQL5\Include\MMAR\PartitionAnalysis.mqh	Мультифрактальный анализ функции разбиения с диагностикой масштабирования (часть 4)
MQL5\Include\MMAR\SpectrumFitter.mqh	Вычисление мультифрактального спектра и подгонка к четырем распределениям методом оптимизации BLEIC (часть 5)
MQL5\Include\MMAR\SimulationEngine.mqh	Модуль моделирования MMAR: мультипликативный каскад, генерация FBM (метод Дэвиса — Харта и разложение Холецкого) и композиция с деформацией времени (часть 6)
MQL5\Include\MMAR\MonteCarlo.mqh	Прогнозирование волатильности методом Монте-Карло: выполняет N симуляций MMAR и вычисляет среднее, медиану, стандартное отклонение и 95%-ный доверительный интервал (часть 7)
MQL5\Scripts\MMAR\MMAR_PartitionTest.mq5	Тестовый скрипт для анализа разбиений на реальных рыночных данных (часть 4)
MQL5\Scripts\MMAR\MMAR_SpectrumTest.mq5	Тестовый скрипт полного пайплайна анализа разбиений и подгонки спектра на реальных рыночных данных (часть 5)
MQL5\Scripts\MMAR\MMAR_SimulationTest.mq5	Тестовый скрипт для проверки инвариантов каскада, свойств FBM и согласованности результатов нескольких прогонов (часть 6)
MQL5\Scripts\MMAR\MMAR_MonteCarloTest.mq5	Сквозной тестовый скрипт: анализ разбиений + подгонка спектра + прогнозирование волатильности методом Монте-Карло (часть 7)

Следующая статья >>

Muhammad Minhas Qamar

[Пакистан](#) [9657](#)

Developer by Profession, Trader by Hobby

Gmail: ayanminhasshayar@gmail.com

Другие статьи автора

- [За пределами GARCH \(Часть VIII\): Библиотека MMAR и ее использование в советнике](#)
- [За пределами GARCH \(Часть VI\): Фракционное броуновское движение и мультипликативный каскад в MQL5](#)
- [За пределами GARCH \(Часть V\): Подгонка мультифрактального спектра в MQL5](#)
- [За пределами GARCH \(Часть IV\): Реализация анализа разбиений в MQL5](#)
- [За пределами GARCH \(Часть III\): Построение MMAR и вердикт](#)
- [За пределами GARCH \(Часть II\): Оценка фрактальной размерности рынков](#)

[Подпишитесь](#) и узнавайте о
популярных статьях и кодах